

```

•   InputComponent->BindAxis("MoveY", this,
&AMyPawn::Move _YAxis);
•
•   }

```

9. The following code gives you the axis function

```

•   void AMyPawn::Move _XAxis(float AxisValue)
•   {
•       // Move at 100 units per second forward or backward
•       CurrentVelocity.X = FMath::Clamp(AxisValue, -1.0f,
1.0f) * 100.0f;
•   }
•
•   void AMyPawn::Move _YAxis(float AxisValue)
•   {
•       // Move at 100 units per second right or left
•       CurrentVelocity.Y = FMath::Clamp(AxisValue, -1.0f,
1.0f) * 100.0f;
•   }

```

10. Growing function

```

•   void AMyPawn::StartGrowing()
•   {
•       bGrowing = true;
•   }
•
•   void AMyPawn::StopGrowing()
•   {
•       bGrowing = false;
•   }

```

11. Compile and add your newly created VR pawn to your project

If you feel the C++ is exhausting and vague then another easier way to create a pawn is with node design.

Creating a VR pawn through nodes

Although programming is great, creating with nodes is much easier and would avoid creating conflicts in movement. The following are the steps to follow:

1. Delete the player start actor and drag a new blank pawn from the sidebar and place it in the center of the scene.